FPGA Accelerator for Directed Percolation

Monte Carlo Simulation on Artix-7 at 100 MHz

Leonardo Pieripolli, Alessio Tuscano

Programmable Hardware Devices

August 21, 2026

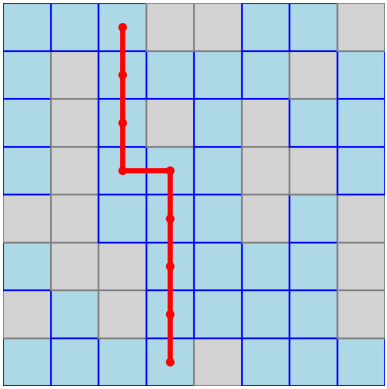
Outline

What is Percolation?

Physical picture: a fluid filters through a porous medium.

- 2D grid: each site is open (blue, prob. p) or closed (gray, $1 - p$)
- A spanning cluster connects top $\rightarrow$ bottom
- Phase transition at critical probability p_c
- Below p_c: only finite clusters
  Above p_c: spanning cluster emerges
- **Isotropic percolation:** fluid can move in all 4 directions

Isotropic Percolation

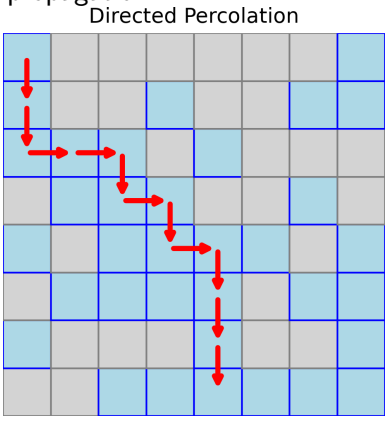


Directed Percolation (DP)

Key difference: flow is directional — only downward propagation.

- Fluid moves only $\downarrow$ (not $\uparrow$)
- Horizontal spread still allowed ($\leftrightarrow$)
- Different universality class from isotropic percolation
- DP critical exponents: useful to find cluster mass and p_c for example

$\beta = 0.2765$	(order parameter)
$\nu_{\parallel} = 1.096$	(corr. length, time)
$\nu_{\perp} = 0.733$	(corr. length, space)



Finite-Size Scaling

For a finite grid of size $N \times N$, the spanning probability obeys:

$$P_{\text{span}}(p, N) = F\left((p - p_c)\, N^{1/\nu}\right)$$

where F is a universal scaling function.

Consequences:

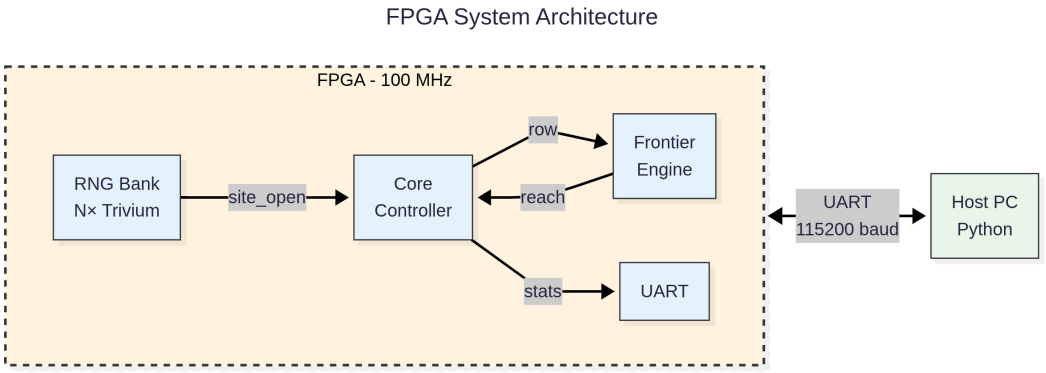
- Curves for different N collapse onto a single master curve
- The collapse validates the universality class
- The crossing point of $P_{\text{span}} = 0.5$ gives an estimate of p_c

Threshold estimation via logistic regression:

$$P_{\text{span}}(p) = \frac{1}{1 + e^{-a(p - p_c)}}$$

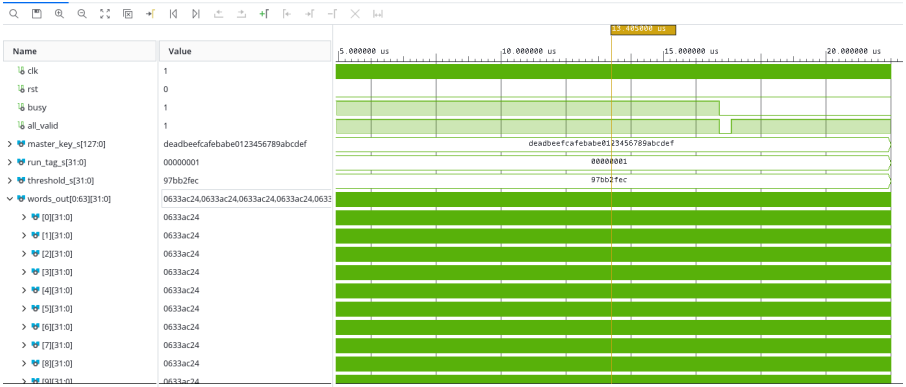
Fitted via maximum likelihood (binomial bootstrap for CIs).

Block Diagram



RNG Bank — Architecture

- N_{rows} independent Trivium stream ciphers, one per grid column
- N_{rows} is a **synthesis-time generic** (`N_ROWS_G`), default 64
- Synthesized for 64, 128, or 180 — same throughput, different resource usage



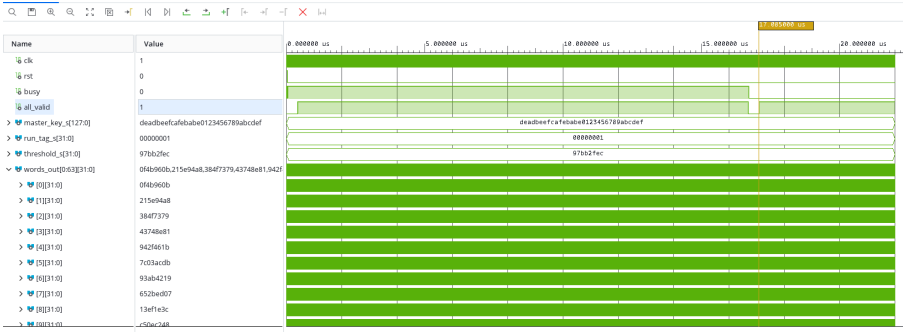
RNG Bank — Seeding & Output

AES-CTR Seeding:

- For $N_{\text{rows}} = 64$: 1536 cycles ($2N$ AES blocks $\times$ 12 cyc)
- Trivium warmup: 36 cycles (1152 bits $\div$ 32)
- Total: $\approx$1573 cycles ($\approx$15.7 μs at 100 MHz)

Per cycle output:

- N_{rows} random 32-bit words

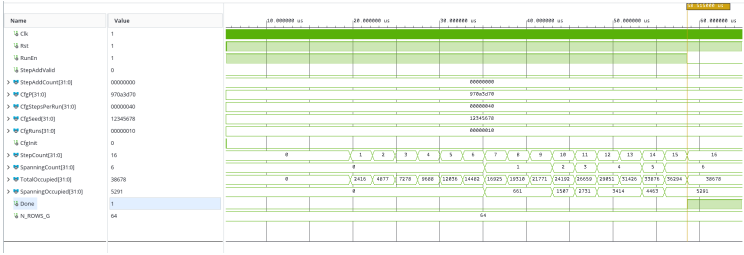


Frontier Engine — Row-Wise Algorithm

Problem: determine if a top-to-bottom path exists without storing the full grid.

Algorithm (per row): i is the *horizontal* position within the row ($0 \leq i < N_{\text{rows}}$).

1. Vertical seed: $\text{seed}[i] = \text{open}[i] \wedge \text{prev\_reach}[i]$
2. Horizontal closure (fixed point):
   $\text{reach}[i] = \text{seed}[i] \vee (\text{open}[i] \wedge (\text{reach}[i-1] \vee \text{reach}[i+1]))$

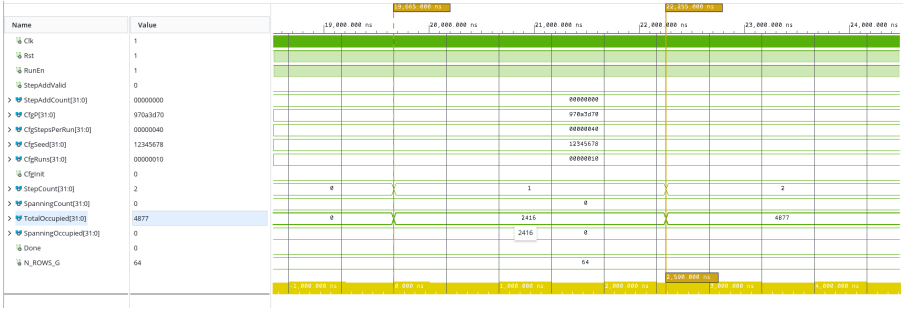


Bidirectional prefix scan: resolves in $\log_2 N_{\text{rows}}$ stages (6 for $N = 64$) in a single cycle

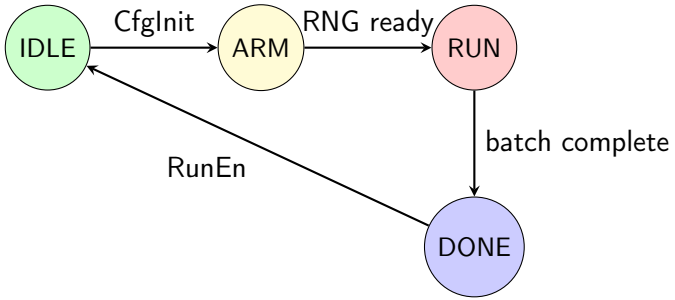
Frontier Engine — VHDL Pipeline

3-stage pipeline:

- RUN_READY → RUN_COMPUTE → RUN_SAVE
- 3 cycles/row core computation
- 4 cycles/row end-to-end (includes registered handshake)
- Only 2 rows of state buffered



Core Controller — State Machine



Batch processing:

- `CfgRuns` runs executed autonomously — no host intervention
- Accumulators (32-bit): StepCount, SpanningCount, TotalOccupied, SpanningOccupied
- RNG runs continuously — no cycle wasted waiting for random numbers

UART Protocol — 16-Byte Frames

Request (host → FPGA):

Word	Field	Purpose
0	CfgP (UQ32)	Probability p
1	CfgSeed	RNG seed
2	CfgStepsPerRun	Grid height
3	CfgRuns	Batch size

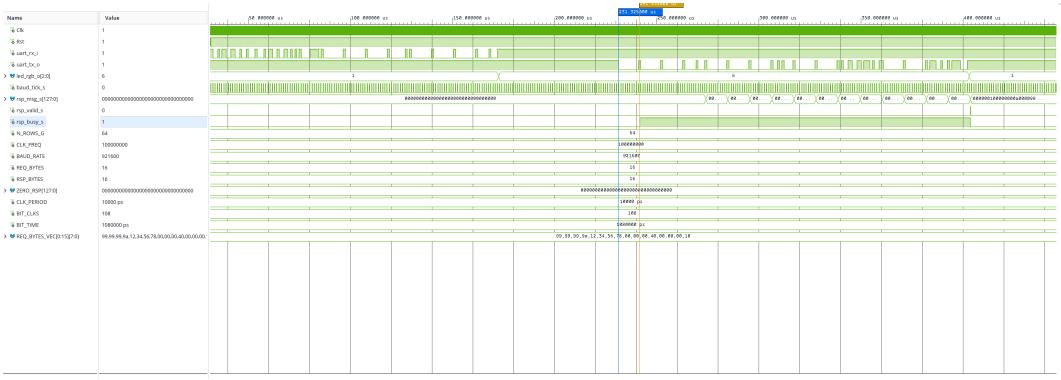
Response (FPGA → host):

Word	Field	Purpose
0	StepCount	Runs completed
1	SpanningCount	Spanning runs
2	TotalOccupied	Total occupied site
3	SpanningOccupied	Reachable sites

Details:

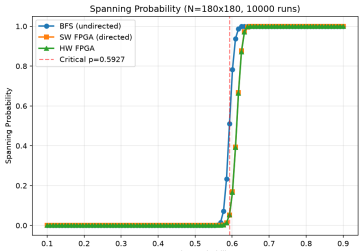
- 115200 baud
- Wire time: $\approx$2.78 ms
- Linux termios raw mode
- Fixed-point: $p = \text{CfgP}/2^{32}$

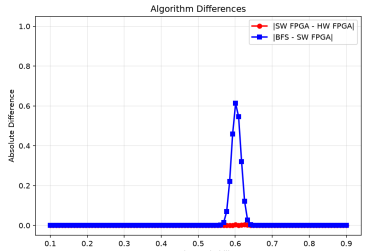
UART Protocol — End-to-End Simulation

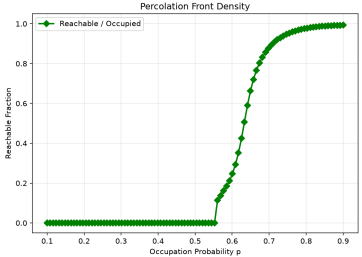


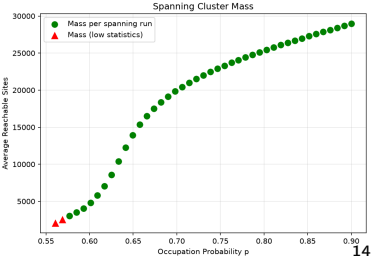
Three-Way Comparison — Full Range

- BFS (undirected) vs SW FPGA (directed) vs HW FPGA
- 20 points, 1000 runs each
- Directed percolation threshold is higher ($p_c \approx 0.605$) than isotropic ($p_c \approx 0.593$)



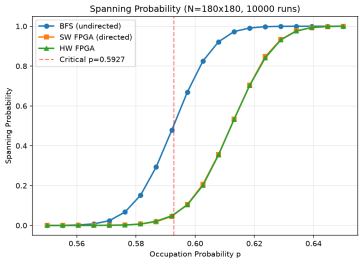


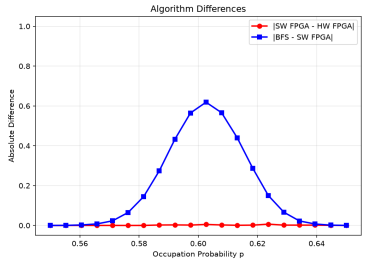


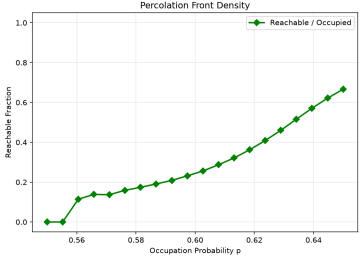


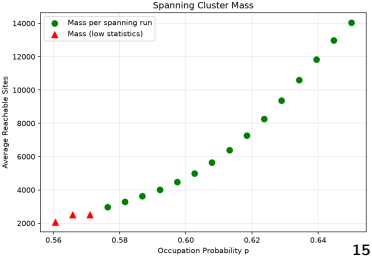
Three-Way Comparison — Threshold Region

- Zoom around the critical region
- 100 points, 1000 runs each
- Perfect agreement between software model and hardware implementation

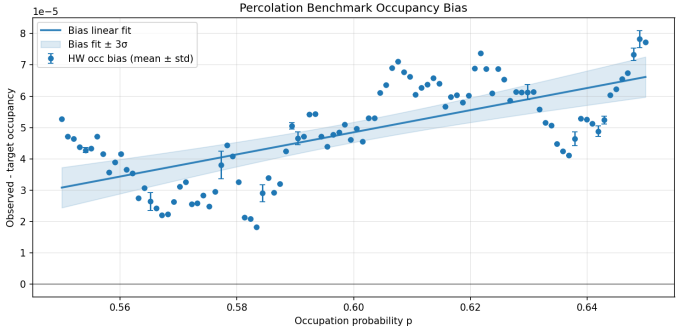








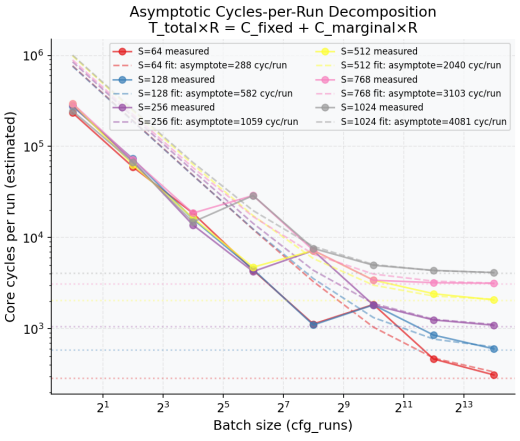
Occupancy Bias — RNG Accuracy

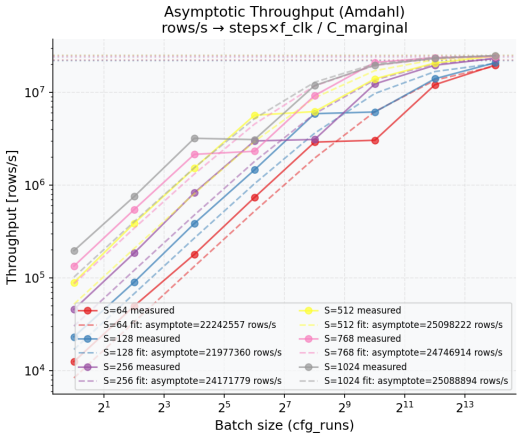


- Bias $= \langle p_{\text{measured}} \rangle - p_{\text{target}}$
- Maximum bias < 0.001 across full p range
- No systematic trend — RNG is unbiased

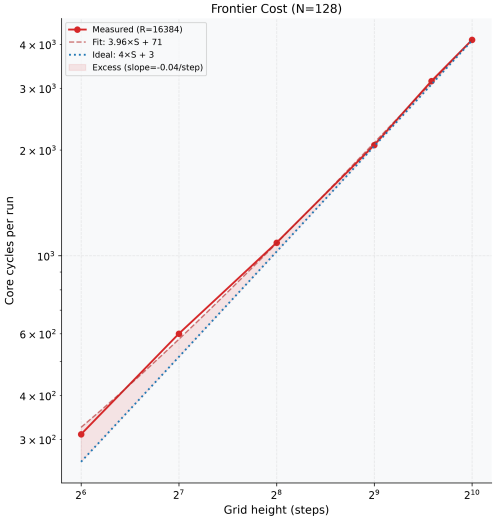
Asymptotic Throughput — Amdahl Decomposition

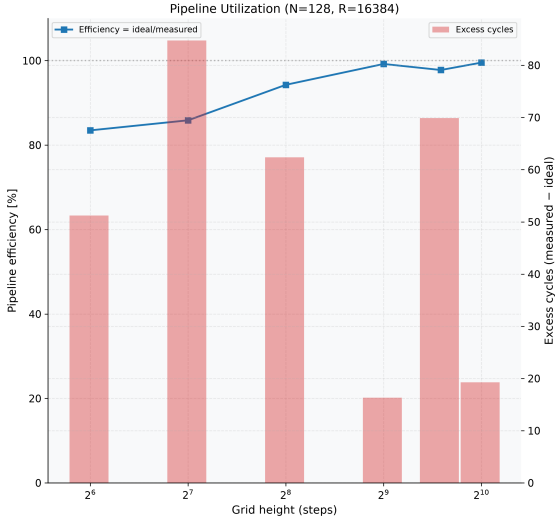
N=128, p≈0.6





Theoretical vs practical efficiency





Model and Speed results

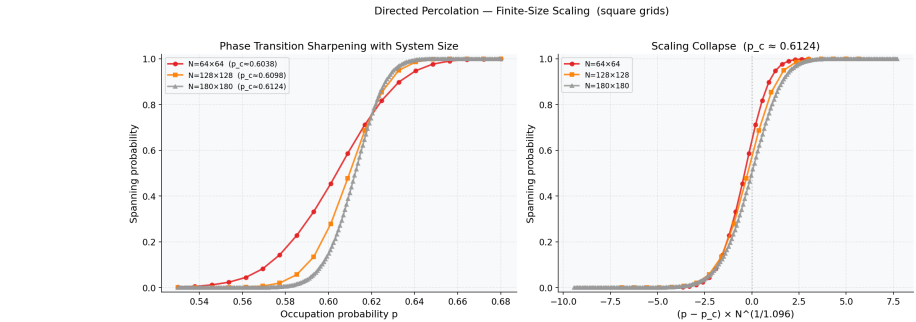
Model: $T_{\text{total}} \times R = C_{\text{fixed}} + C_{\text{marginal}} \times R$

- C_{fixed}: UART + host overhead (one-time per request)
- C_{marginal}: per-run core cost (asymptote at large R)
- **Throughput:** $\dfrac{\text{rows}}{\text{s}} = \dfrac{\text{steps} \times f_{\text{clk}}}{C_{\text{marginal}}}$

Results:

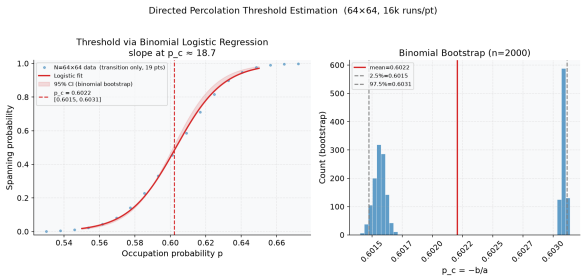
- Asymptote matches theoretical $4 \times S$ cycles/run (3 frontier + 1 handshake)
- Measured: $C_{\text{core}} = 3.96 \times S + \text{const}$ (vs ideal $4 \times S$)
- For $N = 64$: measured asymptote ≈ 288 cycles/run vs theoretical $4N + 3 = 259$ ($\approx$11% state-machine overhead at the latest point we measured)
- Efficiency: $\approx$100% of achievable $4 \times S$ model
- Throughput scales with batch size, saturating at $\sim 2.5 \times 10^7$ rows/s

Finite-Size Scaling — Collapse



- Square grids: $N = 64, 128, 180$
- Perfect collapse using DP exponent $\nu_{\parallel} = 1.096$
- Confirms the system belongs to the directed percolation universality class

Threshold Estimation — Bootstrap



Method: binomial logistic regression + 2000 bootstrap resamples

$$P_{\text{span}}(p) = \frac{1}{1 + e^{-a(p - p_c)}}$$

- $p_c = 0.6022 \pm 0.0007$ (95% CI)
- Fairly consistent with literature value ≈ 0.605 for DP on 64×64

Thank You

Questions?

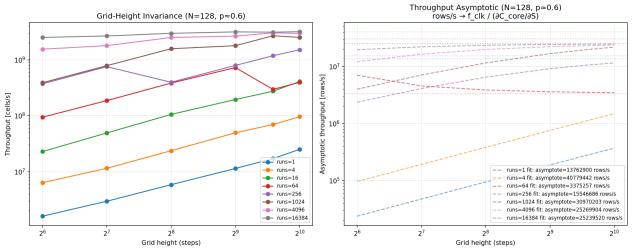
Repository: `https://github.com/tusca99/FPGA-project`

- VHDL: percolation core, RNG bank, frontier engine
- Python: validation, benchmark, analysis
- Full documentation in `project/percolation_core/`

Summary of Results

Metric	Value	Notes
Clock frequency	100 MHz	Single-clock, Artix-7
Grid width N_{rows}	64 (128, 180)	Compile-time generic
Frontier pipeline	3 cycles/row	READY $\rightarrow$ COMPUTE $\rightarrow$ SAVE
Cycles per run ($N \times N$)	$4N + 3$	e.g. $N = 64 \rightarrow \approx 259$
Pipeline efficiency	$\approx$100%	Of achievable $4 \times S$ model
Throughput	$\sim 10^9$ cells/s	At 100 MHz ($\sim 2.5 \times 10^7$ rows/s $\times$ 64)
Occupancy bias	< 0.001	RNG accuracy
Critical threshold p_c	0.6047 ± 0.0002	DP universality class
UART overhead	$\approx$2.78 ms	115200 baud, 32 bytes

Grid-Height Invariance & Determinism



Throughput vs grid height

- Throughput independent of grid height (pipeline-dominated)
- CV $< 1\%$ across all configurations — FPGA is deterministic
- Tiny jitter is entirely host/USB-subsystem noise

Threshold Estimation — Bootstrap